

BotXRGame — Session 4 Progress Log

XR Headset to ROS 2 Link Established on RubikPi 3

Date: 10 August 2026 | Author: swapnil-0

1. Session Goal & Outcome

Goal: Prove a live data link from the Samsung Galaxy XR headset to ROS 2 running on the Qualcomm board. Thumbstick input on the headset should appear as `geometry_msgs/Twist` messages on the `/cmd_vel` topic on the board, with measurable rate and latency.

Outcome: Achieved end to end. Three separate defects were found and fixed along the way — two of them in Unity's own ROS bridge server, which does not work unmodified against the current ROS-TCP-Connector. Measured baseline: **9.0 Hz, 3.1 ms jitter** over Wi-Fi, against 0.9 ms for a local publisher.

2. Environment

Component	Detail
Board	RubikPi 3 (stand-in for Qualcomm Dragonwing IQ-9075)
OS	Ubuntu 24.04 LTS — pairs natively with ROS 2 Jazzy, no Docker
ROS	ROS 2 Jazzy (revised from Humble: 24.04 does not support Humble)
Network	Board wired at 192.168.1.204; headset on Wi-Fi at 192.168.1.102
Headset	Samsung Galaxy XR, Unity app with ROS-TCP-Connector
Access	Passwordless SSH key auth, VS Code Remote-SSH

3. What Was Built

Path	Purpose
<code>ros2_ws/src/xr_link_test</code>	New package. <code>link_monitor</code> node: discovers topics at runtime and prints every inbound message with rate and jitter
<code>ros2_ws/src/ros_tcp_endpoint</code>	Vendored and patched Unity bridge server. See <code>PATCHES.md</code> .

Why `link_monitor` exists. A plain DDS subscriber separates “did the message reach ROS” from “is the bridge configured correctly”. Those two failures look identical from the app side, and telling them apart is what made the rest of the session tractable. It also prints per-topic rate and jitter, which produced the baseline numbers below.

4. Defects Found and Fixed

4.1 Endpoint truncated `syscommand` JSON

Symptom: every connection threw `JSONDecodeError`, the client thread died, and the headset reconnected in a loop. No topic ever registered.

Cause: `message_json = data.decode("utf-8")[:-1]` unconditionally drops the last character, to strip a NUL terminator that older connectors sent. The current connector does not send it, so the closing brace was eaten.

Fix: strip actual NUL bytes instead, which is correct for both protocol generations, and log rather than die on an unparseable payload.

4.2 Endpoint published raw bytes with no CDR header

Symptom: harder — nothing errored. Registration succeeded, the endpoint logged inbound frames at the expected rate, and yet no subscriber ever received anything.

Cause: `RosPublisher.send()` shipped with its deserialization commented out, passing raw bytes to `publish()`. `rcpy` treats bytes as an already-serialized message and pushes them to the wire unvalidated; subscribers then fail to deserialize and drop them *silently*.

Evidence: frame length was 48 bytes. A `Twist` is six `float64` = 48 bytes of payload; a valid CDR message carries a 4-byte encapsulation header, so a correct frame would be 52. The header was missing.

Fix: prepend the little-endian CDR header, deserialize explicitly, publish a real message object, and log failures loudly.

4.3 Unity connect flow tore down its own connection

Symptom: HUD showed green “connected” and the spaceship moved, but Unity sent only registration commands and no data frames.

Cause: `ROSConnection` auto-connects at startup. `ROSIPConfig.OnConnectPressed()` then called `Disconnect()` followed by `Connect()` on that live connection, leaving the sender detached while `HasConnectionError` stayed false.

Fix: stop cycling the connection in the button handler; only set the publish gate.

5. Measured Baseline

Metric	Value	Note
<code>/cmd_vel</code> rate	9.0 Hz	App requested 10 Hz — see 5.1
Jitter over Wi-Fi	3.1 ms	Headset to board
Jitter, local publisher	0.9 ms	<code>ros2 topic pub</code> on the board
Wireless cost	~2.2 ms	Difference attributable to the radio
Frame size	48 bytes	Six <code>float64</code> , no CDR header

5.1 Why 9.0 Hz and not 10

`RobotController.Update()` reset its accumulator to zero after publishing, discarding the overshoot each cycle and rounding the period up to a whole number of frames. At a 72 Hz display: $8 \text{ frames} \times 13.89 \text{ ms} = 111 \text{ ms} =$ exactly 9.0 Hz. Fixed by subtracting the interval instead; publish rate raised to 30 Hz for teleop feel.

6. Architecture Decisions

ROS distro is now Jazzy, project-wide. Ubuntu 24.04 on the board makes Jazzy the native choice. Critically, ROS 2 does not support traffic between distros — a Humble node and a Jazzy node cannot share a DDS graph. This constrains how the Hiwonder JetRover integrates, since it ships with Humble on its own SBC.

Robot platform: Hiwonder JetRover chassis with the Qualcomm board replacing the stock SBC, talking to the onboard STM32 over serial. Lidar, camera and the 6-DOF arm hang off the Qualcomm board.

Custom transport agreed. The XR-to-board link will move off ROS-TCP-Connector to a BotXRGame-owned protocol, with a local gateway publishing into ROS. This session is the strongest argument for it: the upstream

bridge is effectively unmaintained, its wire protocol has drifted from the Unity package it pairs with, and both failure modes above were silent. Owning the transport also makes a later move to QUIC a swap rather than a rewrite.

7. Repository Changes

File	Change
ros2_ws/src/xr_link_test/	New — link_monitor node and package
ros2_ws/src/ros_tcp_endpoint/	New — vendored v0.7.0 with three patches
.../ros_tcp_endpoint/PATCHES.md	New — full rationale for each patch
Assets/Scripts/RobotController.cs	Publish-rate drift fix; rate 10 → 30 Hz
Assets/Scripts/ROSIPConfig.cs	Connect-flow fix
docs/ros-interface/...	Jazzy; corrected install steps; troubleshooting table; baseline
docs/architecture/...	Jazzy; JetRover/IQ-9075 hardware; status updated

Correction to earlier docs: the interface spec previously instructed `sudo apt install ros-humble-ros-tcp-endpoint`. That package has never existed for any ROS 2 distro. It is replaced with build-from-source instructions pointing at the vendored copy.

8. Next Steps

- Design the BotXRGame XR-to-board protocol: message envelope, versioning, framing. Length-prefixed JSON first for debuggability.
- Build the gateway process on the board — owns the socket, publishes to ROS via rclpy.
- Replace ROS-TCP-Connector in Unity with a plain TcpClient against that protocol.
- Re-prove this same milestone over the new transport and compare against the 3.1 ms baseline.
- Bring up the STM32 chassis driver; obtain the reference code from the parallel RubikPi/JetRover effort.
- Lidar publishing /scan and rendering live in the headset.

Verification note: link_monitor logic is covered by 46 assertions run against stubbed ROS interfaces, and the full path was confirmed on hardware on 10 August 2026.